MyCal
version: 0.5

Andi Klein

February 18, 2025

Abstract

MyCal is a program to calculate calories from a database of ingredients. It is based on **psql** and **pyside6**

Contents

1 Introduction **2**

2 Layout of the program **2**

3 class NewWindow **2**

4 class RecipeModel **2**

5 class ControlDB **2**

5.1 Important variables 3
5.2 some poogrammatic issues 3
5.3 ConnectDataBase 3
5.4 SetupLogger 3
5.5 ShowTables 3
5.6 ViewTable 4
5.7 SetPosition 4
5.8 CreateIngredientsForm 4
5.9 InsertRecord 4
5.10 MyRecipes_new 4
5.11 getSelectedText, getSelectedRecipes 4
5.12 FillRecipeIngredientList 5
5.13 DisplayRecipeList 5
5.14 split_and_keep 5
5.15 StripUnicode 5
5.16 GetRecipeList 5
5.17 SizeWindow 6
5.18 SaveIngredients 6
5.19 on_item_click 6
5.20 save_recipe 6
5.21 pack_record 6
5.22 update_recipe 6
5.23 update_record 6
5.24 do_sql 6
5.25 create_pandas_frame 6

6 Some Useful commands **7**

7 Notes **7**

8 Current Status **7**

9 Current issues **7**

10 Design of Calculation **8**

1 Introduction

In the following I will try to descripbe the setup of the program, so that in a few month I still can remember what I did.

2 Layout of the program

Currently, the main class is ControlDB, which has the methods described below. While in the beginning I created the form for ingredients from scratch, I later started to use the QT designer. The advantage of the QT designer is that I can easily add or move things in a design, run the conversion from ui to py and be done. Calling the designer is done by **pyside6-designer Recipe.ui &** to open the Recipe.ui form. After having changed the design and saved it, running **pyside6-uic Recipe.ui -o Recipe.py** converts the design into a python class, which can the be reused.

3 class NewWindow

Creates a simple mainwindow, which can then be used to add widgets.

4 class RecipeModel

This is created from QAbstractListModel and creates the model for the recipe form. This is part of the model view architecture of pyside6 and is used by the method **MyRecipes_new**.

5 class ControlDB

This the main class of the program. It is called by

```
class ContrlDB(QMainWindow):

    def __init__(self,Title=None,db_name=None,db_user=None,db_system=None,db_pwd =
        super().__init__()
        self.db_name = db_name
        self.db_user = db_user
        self.db_system = db_system
        if db_pwd == None:
```

where the database parameters are passed.

5.1 Important variables

1. self.panda_ingredients[][] list of ingredients in a recipe
2. self.MRD.recipes The table entry of the recipes. This contains the list of weights and ingredients
3. self.selected_recipe name of the recipe clicked on

5.2 some poogrammatic issues

The connection to the tableviewmodel goes through **self.RecipeModel.recipes** = []. Defining this creates the data in the model.

5.3 ConnectDataBase

This establishes the connection to the psql database, using a text file for the password. If the connection fails , the program exits

5.4 SetupLogger

This creates the logger system. It deals with warnings and errors.

5.5 ShowTables

Lists all the tables in the database

5.6 ViewTable

This method sets up the connection to a specific table and you can view this table. You can choose the list of columns which you want to suppress in display.

```
def ViewTable(self,table,suppress_columns=[],editmode=True,
columnwidth=[],Title = None, showTable = True):
```

Calling this method with showTable = False will suppress the display. This is one of the main methods to deal with tables and selecting them.

5.7 SetPosition

Move the window to position chosen.

5.8 CreateIngredientsForm

This creates the form to add ingredients. This builds the respective form from scratch, something I don't want to do . It connects to SaveIngredients and also to Cancel

5.9 InsertRecord

First it creates a tableview with the above ViewTable for the ingredients table and then creates a QSqlQuerymodel. It inserts self.record, which has been created by SaveIngredients, which in turn is a connected method called by clicking on the SaveButton from the ingredient form.

5.10 MyRecipes_new

This is the heart of the communication with the recipes table. This establishes the recipe window creates the RecipeView, as well as the RecipeModel. It calls the GetRecipeList, which creates a list of all the recipe names.

5.11 getSelectedText, getSelectedRecipes

This is a function called through a connector in the Recipe Combobox. First it clears the record, and then fills the record with the table row chosen through the combo box. It uses an emit signal to refresh the list.

5.12 FillRecipeIngredientList

This is currently fairly long for historical reasons. The columns , which contains the ingredients of a specific recipe from the original Libreoffice days looks like this:

```
177 g Flour unbleached *u000a79 g Butter *u000a33 g Almond flour *u000a480 g Apric
```

There are a few problems with this:

1. We have spaces in the name of some of the ingredients. I manually changed the ingredients table to have always underscores, but the recipe table is too long.
2. We have characters in which have to do with textformatting (*u000a)
3. It is one string

In this routine we strip out the UTF characters, remove the *, and then create a list in self.recipe_ingredients

Note in vs03 more handling create 2 dimensional list because now we have tableview

5.13 DisplayRecipeList

Adds items to the Recipelist.

5.14 split_and_keep

This is used by FillRecipeIngredientList to separate the long string into a list of values, where the separation character is **g**.

5.15 StripUnicode

removes the u000a from the record.

5.16 GetRecipeList

Gets a list of all the recipes in the recipe table.

5.17 SizeWindow

Creates dimensions of the window selected. it gives the upper corner coordinate and the x and y size.

5.18 SaveIngredients

Connected to from the CreateIngredientsForm, it checks that the new ingredient is unique and then saves the new created ingredient in the table.

5.19 on_item_click

gets connected to QlistView, and called when you click on one of the items.

5.20 save_recipe

Called from save button in recipe form, it saves the current record for ingredients. It then calls pack_record, so that we can update the record.

5.21 pack_record

The record from save_recipe needs to be packed back into a single string.

5.22 update_recipe

called from save recipe it updates the ingredient record.

5.23 update_record

This updates row b with name in table

5.24 do_sql

Executes a sql statement whihc has been formed by the calling method.

5.25 create_pandas_frame

This packs all the pertinent information for CalculateCalories into a pandas frame.

6 Some Useful commands

1. **pyside6-designer Recipe.ui &**
2. **pyside6-uic Recipe.ui -o Recipe.py**

7 Notes

If you change a value in the table form, you have to hit a ¡CR¿; same for entering the name.

It is important to only use underscores in names or ingredients. Do not use spaces. All entries are in **gram**. I find the US system of oz, cups etc totally stone age; so will never use that system.

do **NOT** use plural for any quantity

8 Current Status

I think with QListview for the recipes, I boxed myself into a corner. This is the end of vs 0.2 I create now vs 0.3 where the interface is a tableview. Tableview is working, next step is to be able to edit the fields and then save them.

9 Current issues

- The unpacking of a ingredienst list with the new stored value only gives the first entry. Needs to be solved first. fixed
- finish new recipe setup, current problem is it does not strip the empty records at the end.
- menu

- Populate the textfield in the form like calgram etc
- Create the calorie calculator
- Make sure entries in table get checked for upper and lower characters
- Test for existence in ingredients database. Handle the case where it does not exist.
- Possibly use US govt nutrition database. https://fdc.nal.usda.gov/
- Clean up the code with consistent name scheme.
- ingredient names plural and singular. set all to singular
- Currently it looks like the UPDATE actually deletes the record. This might have to do with my do_sql changes. Need backup of psql table. actually it does not delete it but also not update it. again this might be my do_sql.
- When you add to existing recipe, it bombs on line AttributeError: 'ContrlDB' object has no attribute new_recipe_name

10 Design of Calculation

Before I start, I want to think about the design of the center piece, calculating the calories etc of the recipes. In order to do this, I will need to create a new class. The program as it stands in vs .3 is already unwieldy enough and testing is pretty darn difficult. So this should be a stand alone class.

In the following I will sketch some of the stuff needed.

1. For every recipe I need to access both databases. To get the recipes, ingredients and amounts as well as the values for each ingredient.
2. I will use pandas to deal with the math and storing the variables. Manipulating the variables is more straight forward.
3. I will try to keep this class completely free from pyside6, so that I can keep the code as clean as possible. This will require some kind of interface between the two. This will provide the data from ControlDB to the calculate class. I think the best way is to already fill a data frame

in ControlDB and then pass this dataframe to the CalculateCalories class.

4. In the controlDB, I still need to create a function, which will deal with a missing ingredient. Shall I automatically open the fill ingredient form?

5. Since potentially I would like to use the US nutrition database, I might be able to access a lot more info than just the calories. So I will try to design the class accordingly.

6. In order